

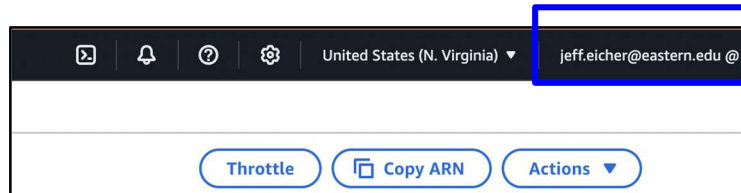


DTSC 671: AI Solutions in the Cloud

SUBMISSION TEMPLATE: Text Extraction with Amazon Textract

Select *File > Make a copy* to edit the template and add your screenshots. Download the completed template as a PDF document to submit to Brightspace.

In every screenshot, include your AWS username that appears in the upper right corner. Example:

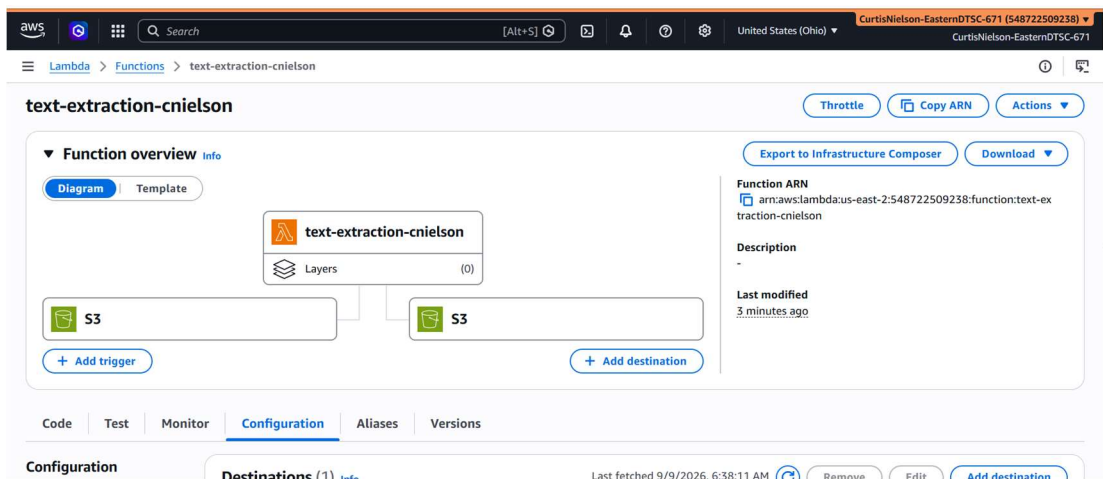


If any one screenshot is missing this identification, the entire assignment will be scored no higher than 70%.

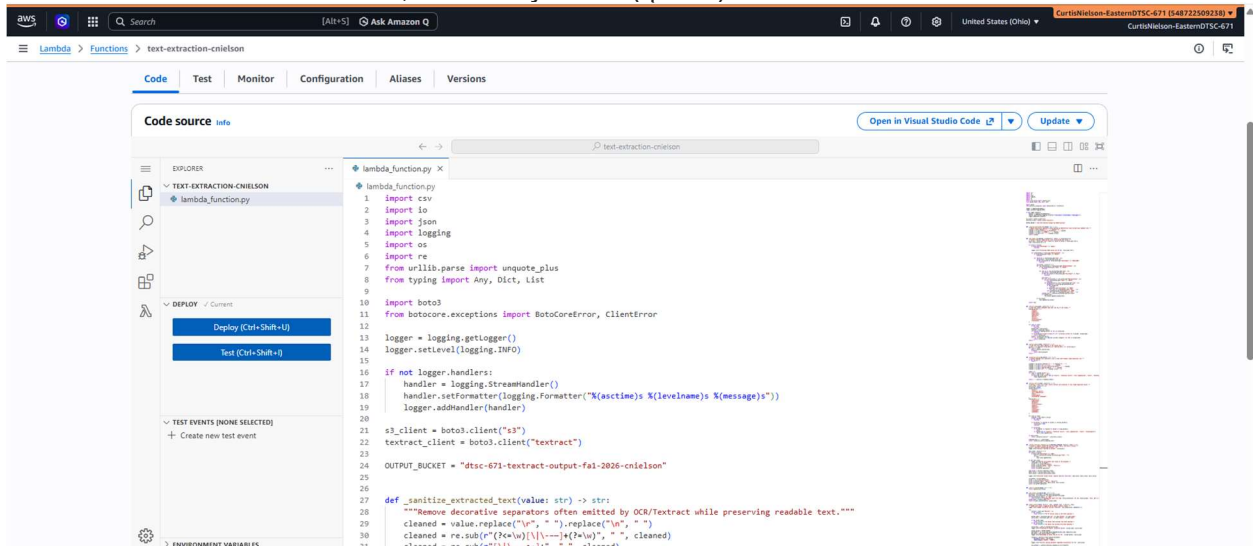
Points totals are provided at the start of each question. You score either full credit or 0 points; there is no partial credit for each question.

Curtis Nielson
DTSC-671

- 1) [10 points] In the space below, provide a screenshot of the Lambda function diagram (with trigger and destination).



- 2) [10 points] In the space below, provide both a screenshot of the beginning of your Lambda code with your account showing, then copy-paste your full lambda code below the screenshot. Note: when writing your function be sure that your code picks up only the skills in the resume, not the symbols (| or -).



```
import csv
import io
import json
import logging
import os
import re
from urllib.parse import unquote_plus
from typing import Any, Dict, List
```

```
import boto3
from botocore.exceptions import BotoCoreError, ClientError
```

```
logger = logging.getLogger()
logger.setLevel(logging.INFO)
```

```
if not logger.handlers:
    handler = logging.StreamHandler()
    handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
    logger.addHandler(handler)
```

```
s3_client = boto3.client("s3")
textract_client = boto3.client("textract")
```

```
OUTPUT_BUCKET = "dtsc-671-textract-output-fa1-2026-cnielson"
```

```
def _sanitize_extracted_text(value: str) -> str:
```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```
"""Remove decorative separators often emitted by OCR/Textextract while preserving readable text."""
```

```
cleaned = value.replace("\r", " ").replace("\n", " ")
cleaned = re.sub(r"(?<=\w)[\|\\\—]+(?>=\w)", " ", cleaned)
cleaned = re.sub(r"[\|\\\—•]+", " ", cleaned)
cleaned = re.sub(r"\s+", " ", cleaned).strip()
return cleaned
```

```
def _get_table_rows(blocks: List[Dict[str, Any]]) -> List[List[str]]:
```

```
    """Convert Textextract TABLE blocks into row-oriented CSV values."""
```

```
    block_map = {block.get("Id"): block for block in blocks if block.get("Id")}
```

```
    rows: List[List[str]] = []
```

```
    for block in blocks:
```

```
        if block.get("BlockType") != "TABLE":
            continue
```

```
        logger.info("Processing TABLE block with ID %s", block.get("Id"))
```

```
        for relationship in block.get("Relationships", []):
```

```
            if relationship.get("Type") != "CHILD":
                continue
```

```
            for child_id in relationship.get("Ids", []):
```

```
                child_block = block_map.get(child_id)
                if not child_block or child_block.get("BlockType") != "TABLE_ROW":
                    continue
```

```
                row_values: List[str] = []
```

```
                for row_relationship in child_block.get("Relationships", []):
```

```
                    if row_relationship.get("Type") != "CHILD":
                        continue
```

```
                    for cell_id in row_relationship.get("Ids", []):
```

```
                        cell_block = block_map.get(cell_id)
                        if not cell_block or cell_block.get("BlockType") != "CELL":
                            continue
```

```
                        cell_text = ""
```

```
                        for cell_relationship in cell_block.get("Relationships", []):
```

```
                            if cell_relationship.get("Type") != "CHILD":
                                continue
```

```
                            for grandchild_id in cell_relationship.get("Ids", []):
```

```
                                grandchild = block_map.get(grandchild_id)
                                if not grandchild:
```

```
                                    continue
```

```
                                if grandchild.get("BlockType") == "WORD":
```

```
                                    cell_text += f"{grandchild.get('Text', '')} "
```

```
                                elif grandchild.get("BlockType") == "LINE":
```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

        cell_text += f"{grandchild.get('Text', '')} "
        cleaned_text = _sanitize_extracted_text(cell_text)
        if cleaned_text:
            row_values.append(cleaned_text)

    if row_values:
        rows.append(row_values)

return rows

def _extract_name(lines: List[str]) -> str:
    """Find the likely candidate name near the top of the resume."""
    heading_markers = {
        "contact",
        "summary",
        "objective",
        "experience",
        "education",
        "skills",
        "projects",
        "certifications",
        "references",
    }

    for line in lines:
        if not line:
            continue
        normalized = line.strip()
        lowered = normalized.lower()
        if lowered in heading_markers or "@" in normalized:
            continue
        if re.fullmatch(r"[A-Za-z][A-Za-z"].-]+(?:\s+[A-Za-z][A-Za-z"].-]+){1,5}", normalized):
            return normalized
        tokens = normalized.split()
        if 2 <= len(tokens) <= 6 and not any(char.isdigit() for char in normalized):
            return normalized
    return ""

def _extract_email(lines: List[str]) -> str:
    """Find the first email address in the resume text."""
    pattern = re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")
    for line in lines:
        match = pattern.search(line)
        if match:
            return match.group(0)
    return ""

```

```

def _normalize_skills(raw_skills: str) -> str:
    """Convert OCR/PDF list separators into a clean CSV-friendly comma-separated list."""
    if not raw_skills:
        return ""

    cleaned = raw_skills.replace("\r", " ").replace("\n", " ")
    cleaned = re.sub(r"\s*[;,|/]+\s*", " ", cleaned)
    cleaned = re.sub(r"\s*[\u2014\u2013]+\s*(?=[A-Za-z0-9])", " ", cleaned)
    cleaned = re.sub(r"\s*\.\s*\.\s*", " ", cleaned)
    cleaned = re.sub(r"\s+", " ", cleaned).strip(" ;")

    items = []
    for item in cleaned.split(","):
        value = item.strip("-|;•/ ")
        if value and value.lower() not in {"skills", "technical skills", "core competencies", "tools",
"technologies"}:
            items.append(value)

    return ", ".join(dict.fromkeys(items))

def _extract_skills(lines: List[str]) -> str:
    """Extract a skill list from a skills section and normalize it into comma-separated values."""
    skill_lines: List[str] = []
    collecting = False
    section_markers = {
        "skills",
        "technical skills",
        "core competencies",
        "tools",
        "technologies",
        "programming languages",
    }
    stop_markers = {
        "experience",
        "education",
        "projects",
        "certifications",
        "summary",
        "objective",
        "contact",
        "references",
        "languages",
        "interests",
    }

    for line in lines:
        lowered = line.lower().strip()
        if not lowered:
            continue

```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

    if any(marker in lowered for marker in section_markers):
        collecting = True
        continue

    if collecting:
        if any(marker in lowered for marker in stop_markers):
            break
        if lowered not in {"skills", "technical skills", "core competencies", "tools", "technologies"}:
            skill_lines.append(line)

    if skill_lines:
        return _normalize_skills(" ".join(skill_lines))

    combined_text = " ".join(lines)
    return _normalize_skills(combined_text)

def _convert_textract_response_to_csv(textract_response: Dict[str, Any]) -> str:
    """Create a compact resume CSV with only Name, Email, and Skills columns."""
    blocks = textract_response.get("Blocks", [])
    logger.info("Textract returned %s blocks", len(blocks))

    text_lines: List[str] = []
    for block in blocks:
        if block.get("BlockType") == "LINE":
            text = _sanitize_extracted_text(block.get("Text", ""))
            if text:
                text_lines.append(text)

    if not text_lines:
        logger.warning("No extractable text found in the document.")
        csv_buffer = io.StringIO()
        writer = csv.writer(csv_buffer)
        writer.writerow(["Name", "Email", "Skills"])
        writer.writerow(["", "", ""])
        return csv_buffer.getvalue()

    name_value = _extract_name(text_lines)
    email_value = _extract_email(text_lines)
    skill_value = _extract_skills(text_lines)

    logger.info("Extracted resume fields: name=%s email=%s skills=%s", name_value,
    email_value, skill_value)

    csv_buffer = io.StringIO()
    writer = csv.writer(csv_buffer)
    writer.writerow(["Name", "Email", "Skills"])
    writer.writerow([name_value, email_value, skill_value])
    return csv_buffer.getvalue()

```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

def _safe_url_decode(value: str) -> str:
    return unquote_plus(value)

def _get_output_key(source_key: str) -> str:
    directory, file_name = os.path.split(source_key)
    base_name, file_extension = os.path.splitext(file_name)
    if file_extension.lower() != ".pdf":
        raise ValueError(f"Unsupported input file type '{file_extension}' for key '{source_key}'. Only
.pdf is supported.")
    output_name = f"{base_name}.csv"
    return os.path.join(directory, output_name)

def lambda_handler(event: Dict[str, Any], context: Any) -> Dict[str, Any]:
    """Lambda entry point triggered by S3 object creation events."""
    logger.info("Lambda invocation started. Event=%s", json.dumps(event, default=str))

    try:
        records = event.get("Records", [])
        if not records:
            raise ValueError("No S3 records found in the event payload.")

        bucket_name = records[0].get("s3", {}).get("bucket", {}).get("name")
        source_key = records[0].get("s3", {}).get("object", {}).get("key")

        if not bucket_name:
            raise ValueError("S3 bucket name missing from event payload.")
        if not source_key:
            raise ValueError("S3 object key missing from event payload.")

        source_key = _safe_url_decode(source_key)
        logger.info("Processing S3 object: bucket=%s key=%s", bucket_name, source_key)

        target_bucket = OUTPUT_BUCKET
        pdf_object = s3_client.get_object(Bucket=bucket_name, Key=source_key)
        pdf_bytes = pdf_object["Body"].read()
        logger.info("Downloaded %s bytes from S3 for %s", len(pdf_bytes), source_key)

        response = textract_client.analyze_document(
            Document={"Bytes": pdf_bytes},
            FeatureTypes=["TABLES", "FORMS"],
        )
        logger.info("Textract analyze_document completed successfully for %s", source_key)

        csv_content = _convert_textract_response_to_csv(response)
        output_key = _get_output_key(source_key)

```

```

s3_client.put_object(
    Bucket=target_bucket,
    Key=output_key,
    Body=csv_content.encode("utf-8"),
    ContentType="text/csv",
)

logger.info(
    "CSV uploaded successfully to S3 bucket=%s key=%s",
    target_bucket,
    output_key,
)

return {
    "statusCode": 200,
    "body": json.dumps(
        {
            "message": "PDF processed successfully via Amazon Textract.",
            "sourceBucket": bucket_name,
            "sourceKey": source_key,
            "outputBucket": target_bucket,
            "outputKey": output_key,
        }
    ),
}

except (ClientError, BotoCoreError) as exc:
    logger.exception("AWS SDK error while processing S3/Textract request: %s", exc)
    return {
        "statusCode": 500,
        "body": json.dumps({"error": "AWS service error", "details": str(exc)}),
    }

except ValueError as exc:
    logger.exception("Validation error: %s", exc)
    return {
        "statusCode": 400,
        "body": json.dumps({"error": "Validation error", "details": str(exc)}),
    }

except Exception as exc: # pragma: no cover - final safety net
    logger.exception("Unexpected error occurred in Lambda handler: %s", exc)
    return {
        "statusCode": 500,
        "body": json.dumps({"error": "Unexpected processing error", "details": str(exc)}),
    }

```

- The screenshot displays the AWS CloudWatch console interface. At the top, the breadcrumb navigation shows 'CloudWatch' > 'Log management' > '/aws/lambda/text-extraction-cnslon'. The left-hand navigation pane includes sections like 'Favorites and recent', 'Ingestion', 'Dashboards', 'Alarms', and 'AI Operations', with 'GenAI Observability' currently selected. The main content area is divided into three columns:

 - Left Column:** Metadata for the log group 'extraction-cnslon:'. It lists 'Creation time' as 2 hours ago, 'Retention' as 'Never expire', 'Stored bytes' as '-', and 'Metric filters' as 0.
 - Middle Column:** Security settings including 'KMS key ID' (-), 'Deletion protection' (OFF), 'Data protection' (-), and 'Sensitive data count' (-).
 - Right Column:** Integration status for 'Anomaly detection' (with a 'Configure' link), 'Bearer token authentication' (OFF), and 'Syslog ingestion' (with links for 'Info' and 'Configure').
 Below this metadata section is a horizontal tab bar with options: 'Log streams' (selected), 'Tags', 'Data protection', 'Anomaly detection', 'Metric filters', 'Subscription filters', 'Contributor Insights', 'Field indexes', and 'Transformer'. The 'Log streams (7)' section follows, featuring a search filter 'Filter log streams or try prefix search', a dropdown menu set to 'Exact match', and checkboxes for 'Show expired' and 'Info'. A table below lists seven log streams, each with a checkbox, a truncated log stream name (e.g., '2026/09/09/[S[LATEST]]2b6561ae0ee4739a28bf3630a9bad9'), and its corresponding 'Last event time' (ranging from 12:31:00 UTC to 10:40:58 UTC). At the bottom left, there is a link for 'Observability Workshops'.

- The screenshot displays the AWS CloudWatch Log Groups console. The breadcrumb navigation shows the path: CloudWatch > Log management > /aws/lambda/text-extraction-critique. The main heading is 'Log events'. Below this, a search bar is present with the placeholder text 'Filter events - press enter to search'. To the right of the search bar are buttons for 'Actions', 'Start tailing', and 'Create metric filter'. Below the search bar, there are tabs for 'Timestamp' and 'Message'. The log events are listed in a table with columns for 'Timestamp' and 'Message'. The messages contain details about the Lambda function's execution, including the runtime version, environment variables, and the results of the text extraction process.

Timestamp	Message
2023-09-07T12:18:58.887Z	[INFO] START Runtime version: python3.14.mainline.v66 Runtime version ARN: arn:aws:lambda:us-east-1:runtime:us6877f6c0db26851e670b24862926c8b0c4c1231c1347c8230ef9191
2023-09-07T12:18:59.422Z	[INFO] RequestId: 69366362-6480-4540-9361-d8a0d05f86c9 Found credentials in environment variables:
2023-09-07T12:13:00.803Z	START RequestId: 69366362-6480-4540-9361-d8a0d05f86c9 Version: SLATEST
2023-09-07T12:13:00.803Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Lambda Invocation started. Event:{"Records":[{"eventVersion": "2.0", "eventSource": "aws:iot", "messageId": "f41-e8-2023-09-07T12:13:00.830Z"}]}
2023-09-07T12:13:00.830Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Processing S3 object: bucket:aws-iot-text-extraction-key-resume_for_textextract.pdf
2023-09-07T12:13:00.830Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Downloaded 18461 bytes from S3 for resume_for_textextract.pdf
2023-09-07T12:13:00.830Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Textextract_analyze_document completed successfully for resume_for_textextract.pdf
2023-09-07T12:13:00.830Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Textextract returned 226 blocks
2023-09-07T12:13:00.830Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Textextract returned fields: name=John Doe email=John.Doe@example.com skills=AWS Lambda, EC2, S3, AWS, dynamoDB Python, Node.js, Bash Terraform, C...
2023-09-07T12:13:00.830Z	[INFO] 2023-09-07T12:13:00.830Z 69366362-6480-4540-9361-d8a0d05f86c9 Csr uploaded successfully to s3 bucket:aws-iot-text-extraction-key-resume_for_textextract.pdf
2023-09-07T12:13:00.830Z	END RequestId: 69366362-6480-4540-9361-d8a0d05f86c9
2023-09-07T12:13:00.830Z	REPORT RequestId: 69366362-6480-4540-9361-d8a0d05f86c9 Duration: 3760.76 ms Billed Duration: 4000.00 ms Memory Size: 128 MB Max Memory Used: 100 MB Init Duration: 1147.27 ms
2023-09-07T12:13:00.830Z	No newer events at this moment. Auto retry pending. Resume

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

- 5) [10 points] In the space below, provide a screenshot of IAM role permissions for the Lambda function.

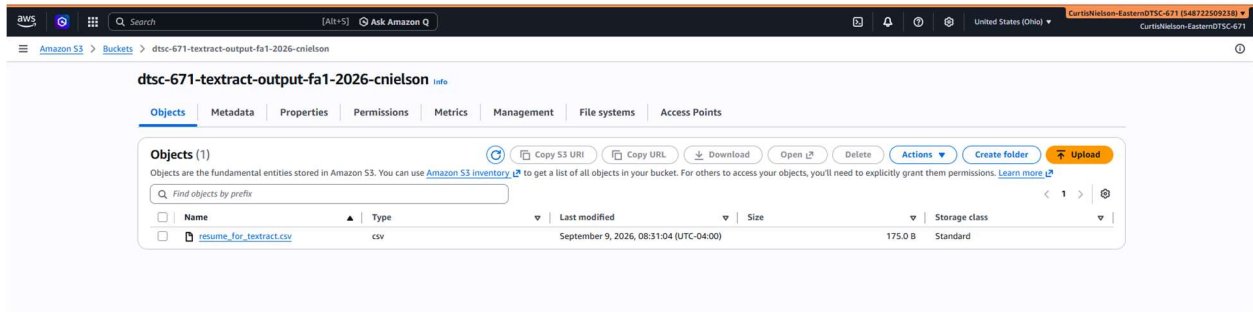
The top screenshot shows the AWS Lambda console configuration page for the function 'text-extraction-cnielson'. The 'Execution role' is set to 'text-extraction-cnielson-role-rmq4j8z4'. The 'Resource summary' section shows the role's ARN and maximum session duration of 1 hour.

The bottom screenshot shows the IAM console 'Permissions' tab for the role 'text-extraction-cnielson-role-rmq4j8z4'. It lists three permissions policies:

Policy name	Type	Attached entities
AmazonS3FullAccess	AWS managed	5
AmazonTextractFullAccess	AWS managed	1
AWSLambdaBasicExecutionRole-ea930ac4-46d4-4072-835f-526...	Customer managed	1

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

- 6) [25 points] In the space below, provide a screenshot of the CSV file in your output bucket.



- 7) [25 points] Provide a screenshot of your CSV file's output (which extracted fields). Fit all of the information in the screenshot.

